

THE ISLAND

Rapport Final de Projet

Projet Programmation Orientée Objet
IATIC 3 - 2025/2026

Rayen HMIDA

Yassine Joudi

Ahmed Ishagh

Mouhamed EL OUARGULY

Encadrante : Dhekra ABOUDA

Table des matieres

Table des matieres	1
1. Introduction	1
1.1 Objectifs du projet	1
1.2 Perimetre du projet.....	1
2. Architecture et conception	1
2.1 Patron MVC.....	1
2.2 Diagramme de packages	1
2.3 Conception du modele	1
Coordonnees hexagonales (HexCoord)	1
Plateau de jeu (Board).....	1
ActionResult - patron Value Object.....	1
3. Implementation	1
3.1 Regles de jeu implementees	1
3.2 Interface graphique	1
Rendu hexagonal	1
Phases de jeu et boutons.....	1
4. Manuel utilisateur.....	1
4.1 Lancer le jeu.....	1
4.2 Configuration de la partie.....	1
4.3 Deroulement d'une partie.....	1
Phase de mise en place	1
Tour de jeu.....	1
Symboles sur le plateau	1
4.4 Fin de partie	1
5. Statistiques du projet	1
5.1 Metriques du code.....	1
5.2 Repartition des taches	1
5.3 Difficultes rencontrees	1
5.4 Pistes d'amelioration	1
6. Conclusion	1
Annexe : Commandes utiles	1

1. Introduction

Ce document constitue le rapport final du projet de Programmation Orientee Objet (IATIC 3, annee 2025/2026). L'objectif du projet etait de concevoir et developper une version informatique jouable du jeu de societe The Island, en utilisant le langage Java et la bibliotheque graphique Swing.

The Island est un jeu de strategie pour 2 a 4 joueurs dans lequel chaque joueur doit evacuer ses dix explorateurs d'une ile en train de sombrer, tout en cherchant a maximiser la valeur des tresors transportes par ceux qui survivent.

1.1 Objectifs du projet

Le projet devait satisfaire plusieurs exigences :

- Implementer les regles completes du jeu de societe The Island
- Proposer une interface graphique claire et ergonomique en Swing
- Respecter le paradigme Objet (encapsulation, polymorphisme, patterns de conception)
- Produire un code documentee (Javadoc) et conforme aux conventions Java
- Fournir un JAR executable autonome buildable hors de tout IDE

1.2 Perimetre du projet

L'implementation couvre l'integralite des regles specifiees dans le sujet :

- Plateau hexagonal avec 40 tuiles terrain (plage, foret, montagne)
- Gestion des 4 types de pions : explorateurs, bateaux, creatures marines
- Toutes les actions de tuiles (immediates et conservees)
- Les 3 creatures (serpent, requin, baleine) avec leurs effets specifiques
- Systeme de score et ecran de fin de partie
- Interface d'aide integree avec resume des regles

2. Architecture et conception

2.1 Patron MVC

L'architecture globale du projet suit le patron Modele-Vue-Contrôleur (MVC). Ce choix garantit une séparation nette des responsabilités et facilite la maintenance.

Couche	Packages	Role
Modele	theisland.model, theisland.util	Données de jeu, logique métier pure. Aucune dépendance vers Swing.
Vue	theisland.view, theisland.view.component	Rendu graphique Swing. Aucune logique de jeu.
Contrôleur	theisland.controller	Validation et application des actions. Pont entre modèle et vue.

2.2 Diagramme de packages

L'organisation des packages respecte strictement la séparation MVC :

```
theisland/  
  Main.java                # Point d'entree  
  util/HexCoord.java       # Coordonnees hexagonales  
  model/                   # Modele de donnees  
    Board.java, Cell.java, GameState.java, Player.java ...  
  pion/ Explorer, Boat, Creature ...  
  tile/ TerrainTile, TileAction, TerrainType ...  
  controller/  
    GameController.java, ActionResult.java  
  view/  
    GameWindow.java, SetupDialog.java, ScoreDialog.java, HelpDialog.java  
    component/ BoardPanel, HexRenderer, PlayerInfoPanel, ActionBarPanel
```

2.3 Conception du modèle

Coordonnées hexagonales (HexCoord)

La classe HexCoord encapsule les coordonnées en layout offset-column. Le calcul des 6 voisins tient compte de la parité de la colonne. La distance hexagonale est calculée en convertissant en coordonnées cubiques.

Plateau de jeu (Board)

Le plateau est représenté par une Map<HexCoord, Cell> permettant un accès O(1) par coordonnée. La zone île est déterminée par une ellipse paramétrique centrée sur le plateau. Les 4 coins accueillent les plages de débarquement.

ActionResult - patron Value Object

Toutes les methodes du controleur retournent un ActionResult contenant : le statut (succes/erreur), un message principal et une liste d'effets secondaires. Ce patron evite de melanger la logique dans les callbacks de la vue.

Classe	Role	Points cles
GameState	Etat global de la partie	Cree les 40 tuiles, gere les tours, calcule les scores
Board	Plateau hexagonal	Map de cases, validation du retrait des tuiles, localisation des pions
Cell	Case hexagonale	Contient explorateurs/nageurs/bateaux/creatures
GameController	Logique de jeu	Valide et applique chaque action, gere les effets en chaine
Explorer	Pion explorateur	Valeur de tresor cachee, etat (terre/bateau/nageur/sauve/elimine)
Boat	Pion bateau	Capacite 3, calcul du controleur par majorite de passagers
Creature	Pion creature	3 types avec comportements distincts (serpent/requin/baleine)

3. Implementation

3.1 Regles de jeu implementees

L'ensemble des regles specifiees dans le sujet a ete implemente. Le tableau ci-dessous recense les principales regles et leur localisation dans le code :

Regle	Classe implementant la regle
Placement initial (explorateurs + bateaux)	GameWindow.setupPlaceExplorer/Boat
3 deplacements par tour	GameState.movesRemaining, GameController.moveExplorer/Boat
Ordre retrait : Plage > Foret > Montagne	Board.canRemoveTile()
Tuile adjacente a la mer pour retrait	Board.canRemoveTileIgnoreOrder()
Actions immediates (requin, baleine, bateau, tourbillon, volcan)	GameController.applyImmediateAction()
Tuiles conservees (dauphin, vent, deplacer creature, chasser)	GameController.playSavedTile()
Effets du serpent de mer (coule bateau + tue nageurs)	GameController.applySeaSnakeEffect()
Effets du requin (tue nageurs uniquement)	GameController.applySharkEffect()
Effets de la baleine (chavire bateau, tue si requin present)	GameController.applyWhaleEffect()
Tourbillon (retire tout dans la zone)	GameController.applyTourbillon()
Controle d'un bateau par majorite	Boat.getControllers()
Score final (somme des tresors des explores sauves)	Player.computeScore()
Egalite de score (nb explores sauves)	GameState.computeWinner()

3.2 Interface graphique

Rendu hexagonal

La classe HexRenderer calcule les positions pixel de chaque case (flat-top, offset-column). Le rendu utilise Java2D avec anti-aliasing. La methode pixelToHex() effectue la detection inverse par recherche du centre le plus proche.

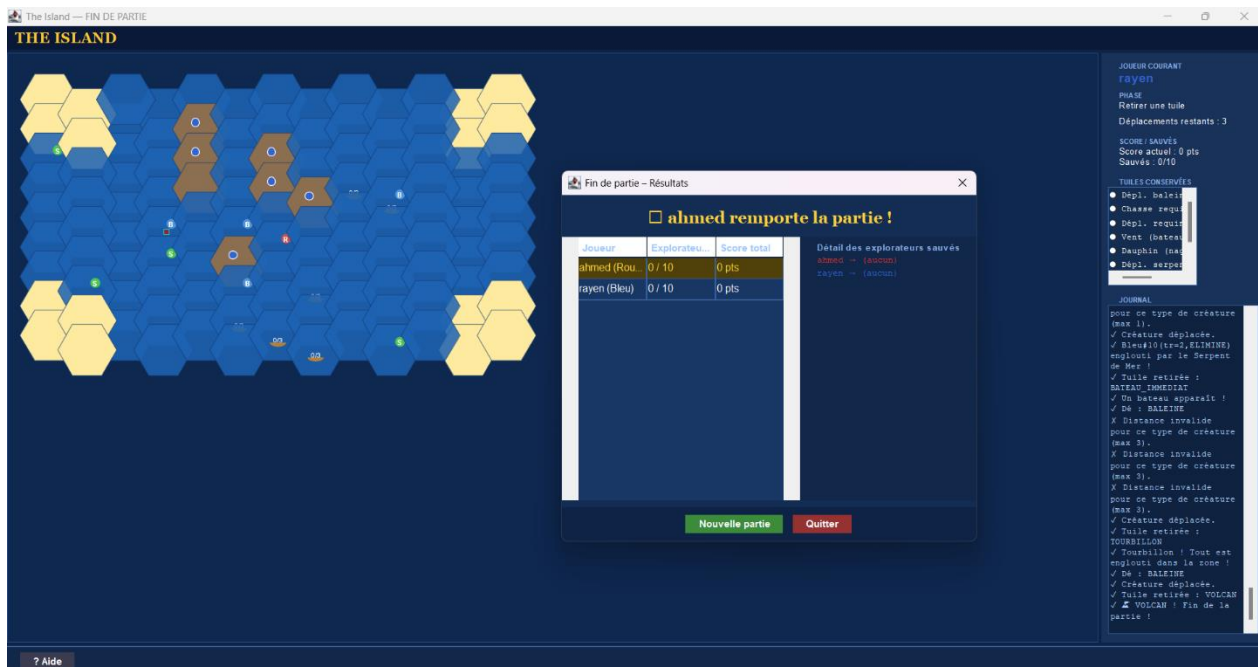
ESPACE RESERVE : capture d'ecran du plateau en cours de partie



Phases de jeu et boutons

La barre d'actions (ActionBarPanel) affiche uniquement les boutons pertinents pour la phase courante. Le passage de phase est orchestré par GameWindow qui agit comme médiateur entre les composants graphiques et le contrôleur.

ESPACE RESERVE : capture d'écran de la boîte de dialogue de fin de partie



4. Manuel utilisateur

4.1 Lancer le jeu

Pre-requis : Java 17 ou superieur installe sur le poste.

Methode 1 - JAR executable :

```
java -jar the-island.jar
```

Methode 2 - Compilation depuis les sources :

```
javac -d target/classes $(find src -name "*.java")  
java -cp target/classes theisland.Main
```

4.2 Configuration de la partie

Au demarrage, une boite de dialogue permet de :

1. Choisir le nombre de joueurs (2 a 4) avec le compteur
2. Saisir le nom de chaque joueur dans les champs prevus
3. Cliquer sur 'Demarrer la partie'

4.3 Deroulement d'une partie

Phase de mise en place

Chaque joueur place ses 10 explorateurs un par un sur les tuiles terrain, puis ses 2 bateaux sur des cases de mer adjacentes a une tuile. Clic gauche sur une case pour placer le pion courant.

Tour de jeu

Chaque tour comprend 4 etapes effectuees dans l'ordre :

4. (Optionnel) Jouer une tuile conservee via le bouton 'Jouer tuile conservee'
5. Deplacer les pions : cliquer sur un pion pour le selectionner, puis sur la destination. Le compteur de deplacements restants s'affiche en haut a droite. Cliquer sur 'Fin des deplacements' quand vous avez termine.
6. Retirer une tuile : cliquer directement sur la tuile a retirer. L'ordre plage > foret > montagne est verifie automatiquement.
7. Lancer le de : cliquer sur 'Lancer le de', puis selectionner une creature du type indique et cliquer sur sa case de destination.

Symboles sur le plateau

Symbole	Signification
Cercle colore (plein)	Explorateur sur une tuile terrain
Carre colore (bord cyan)	Nageur dans l'eau
Arc brun + chiffre	Bateau avec ses passagers
S vert	Serpent de mer

R rouge	Requin
B bleu	Baleine
Cases jaunes (coins)	Plages de débarquement (zones sûres)

4.4 Fin de partie

La partie se termine quand la tuile Volcan est révélée. Un écran de scores affiche le classement de tous les joueurs avec le détail des explorateurs sauvés et leurs valeurs de trésor. Il est possible de lancer une nouvelle partie depuis cet écran.

5. Statistiques du projet

5.1 Metriques du code

Metrique	Valeur
Nombre de classes Java	26
Lignes de code totales	~4 150
Packages	6 (util, model, model.pion, model.tile, controller, view, view.component)
Classes du modele	13
Classes de la vue	8
Classes du controleur	2
Enumerations	6 (TerrainType, TileAction, GamePhase, PlayerColor, CreatureType, ExplorerState)

5.2 Repartition des taches

Ce tableau presente la repartition indicative des taches dans l'equipe (a adapter selon la repartition reelle de votre groupe) :

Membre	Taches principales
ahmed	Modele de donnees (Board, Cell, GameState), coordonnees hexagonales
yassine	Controleur (GameController, ActionResult), regles de jeu
rayen	Interface graphique (BoardPanel, HexRenderer, rendu hexagonal) Test, readme, rapport finale
mohamed	Fenetres et dialogues (GameWindow, SetupDialog, ScoreDialog, HelpDialog)

5.3 Difficultes rencontrees

- Calcul du voisinage hexagonal avec offset-column : la parite de la colonne change les decalages, ce qui a necessite une attention particuliere et des tests.
- Gestion des effets en chaine (ex : baleine sur case avec requin) : l'ordre d'evaluation des effets etait crucial pour eviter des etats incoherents.
- Phase de setup multi-joueur : l'orchestration du placement de 10 explorateurs + 2 bateaux par joueur dans la vue a demande une machine a etats dediee.
- Detection du clic hexagonal : la fonction pixelToHex par recherche du centre le plus proche s'est averee plus robuste que les formules algebriques exactes (qui souffrent d'erreurs de virgule flottante aux frontieres).

5.4 Pistes d'amelioration

- Sauvegarde et chargement d'une partie (serialisation)
- Mode IA pour jouer seul contre l'ordinateur
- Animations lors des déplacements et des effets de creatures
- Son et musique d'ambiance
- Zoom et scroll sur le plateau pour les petits ecrans

6. Conclusion

Ce projet nous a permis de mettre en pratique les principes de la programmation orientee objet dans un contexte applicatif concret et ludique. L'architecture MVC adoptee a facilite la collaboration au sein de l'equipe en permettant a chacun de travailler sur sa couche sans bloquer les autres.

Le jeu The Island presentait une complexite methodologique interessante : la logique hexagonale, les effets en chaine des creatures, l'ordre de retrait des tuiles, et la gestion du controle des bateaux etaient autant de problemes qui ont exige une reflexion approfondie avant de coder.

Le resultat est une application jouable, documentee et maintenable, livree sous forme de JAR executable conforme aux exigences du sujet.

Annexe : Commandes utiles

Compiler le projet :

```
find src -name "*.java" > sources.txt
javac -d target/classes @sources.txt
```

Creer le JAR :

```
echo "Main-Class: theisland.Main" > MANIFEST.MF
jar cfm the-island.jar MANIFEST.MF -C target/classes .
```

Generer la Javadoc :

```
javadoc -d doc -sourcepath src/main/java -subpackages theisland
```